

Plataforma de expositores y credenciales

Administra las empresas que exponen en la feria y las credenciales del personal que opera sus stands. La organización ve, stand por stand, cuántas credenciales hay asignadas y cuántas quedan; cada representante acredita a su gente —una a una o subiendo un Excel— dentro del cupo que le da su metraje, e imprime los gafetes desde la misma pantalla.

12

TABLAS, 2 DE CONFIG.

0

CIFRAS EN EL CÓDIGO

3 / 3

PUNTOS EXTRA

1

COMANDO PARA ARRANCAR

EL EJE DEL DISEÑO

El enunciado pide que se puedan crear ferias futuras **sin que los datos se mezclen**. Eso no es una nota al pie: es lo que ordena el modelo. Toda tabla operativa lleva `event_id`, ningún repositorio se puede construir sin él, y ese `event_id` sale siempre del token, nunca de la petición.

La segunda columna vertebral es la **parametrización**: rangos de metraje, cuotas por categoría y hasta el criterio de redondeo viven en tablas. En el repositorio no existe ningún `if m2 > 12`.

ALCANCE ENTREGADO

Organización

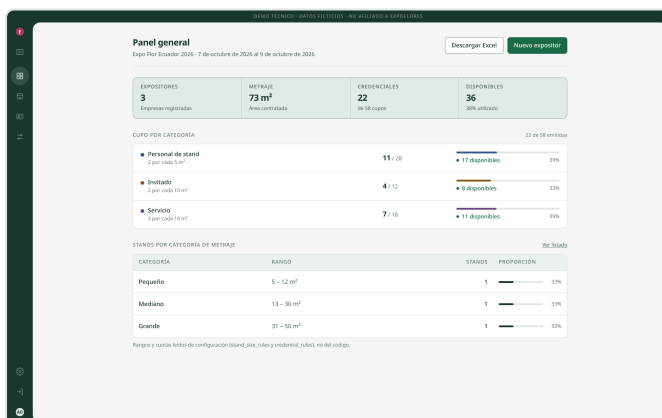
CRUD de expositores con alta atómica de la cuenta del representante, panel de asignadas vs. disponibles, reporte en Excel, reenvío del enlace de acceso.

Representante

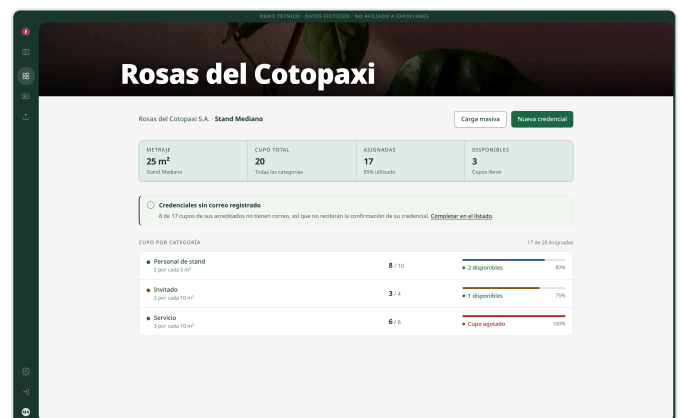
Panel de cupo por categoría, alta y baja de credenciales, carga masiva desde Excel con vista previa e impresión de gafetes.

Puntos extra

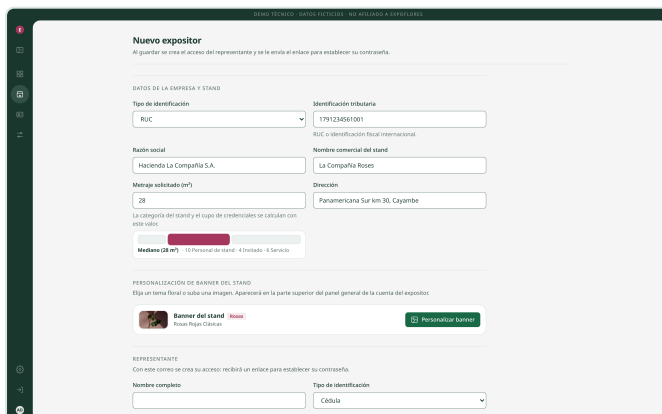
Los tres: correos transaccionales, lectura del Excel en el navegador y reglas parametrizadas en base de datos.



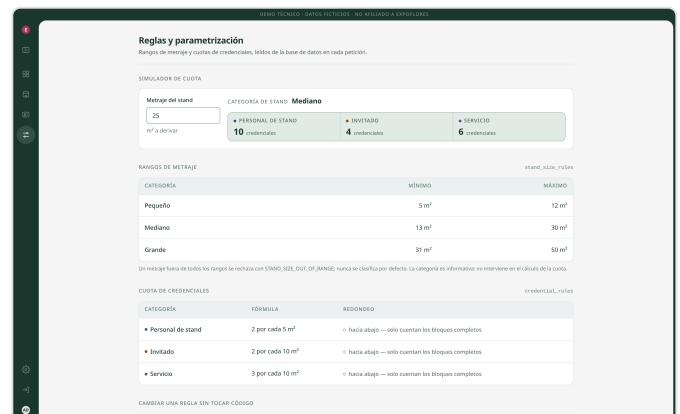
Panel de la organización. Cupo de toda la feria y reparto de stands por categoría. Los rangos no están escritos en la vista: se leen de `stand_size_rules`.



Panel del representante. Cupo por categoría de su stand, calculado en cada lectura. El aviso de credenciales sin correo nace de la decisión 8.



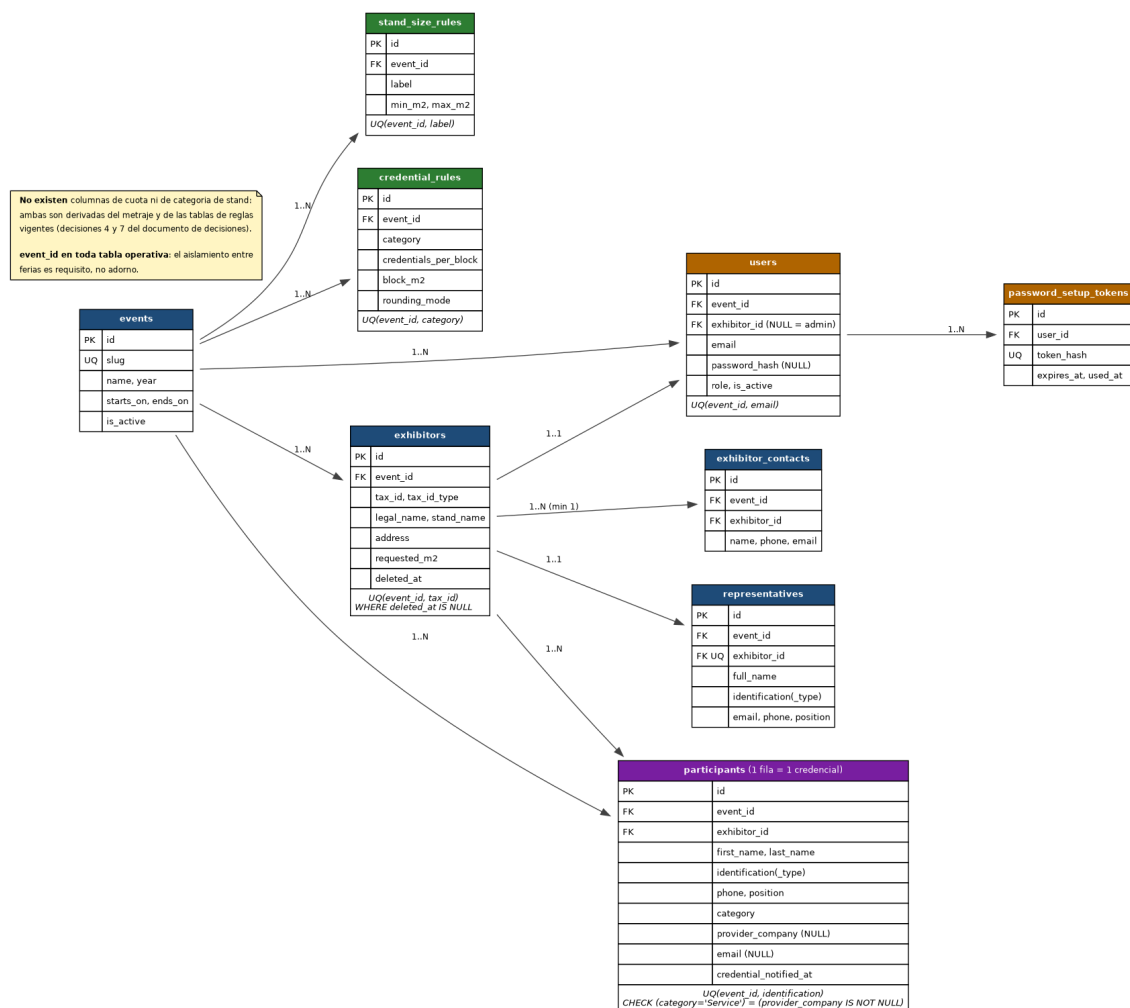
Alta de expositor. Una transacción crea empresa, representante, contactos, cuenta y token. El metraje anticipa la categoría y el cupo.



Reglas y parametrización. Las dos tablas de configuración, leídas de la base en cada petición, con un simulador que reusa la función del alta real.

Modelo de datos

Doce tablas, dos de ellas de configuración. Ninguna cifra de negocio es una columna calculada ni un valor congelado.



Por qué el scope por evento está en todas partes

- **Toda tabla operativa lleva event_id**, incluso donde sería deducible por la relación padre. Deducirlo obliga a un JOIN para no filtrar datos; tenerlo permite que la condición esté siempre en el WHERE.
- **Las claves únicas son compuestas, nunca simples.** UNIQUE(event_id, identification) implementa la validación crítica y a la vez permite que la misma persona vuelva en la edición siguiente con otra empresa. Una restricción global habría bloqueado registros legítimos.
- **El repositorio no se puede construir sin event_id.** No es comodidad: es la barrera que impide que llegue a existir un método de listado sin scope, que es por donde se cuelan las fugas entre ferias y los IDOR.
- **No hay columna de cuota ni de categoría de stand.** Ambas se derivan del metraje y de las reglas vigentes en cada lectura. Una columna congelada se desincroniza en cuanto alguien corrige un metraje.
- **Borrado lógico solo en expositores.** Borrar en duro una empresa con credenciales emitidas es destructivo. Los participantes sí se borran de verdad: borrar una credencial libera cupo y libera la identificación.
- **password_hash es nutable** hasta que el representante establece su clave con el token de un solo uso. Un usuario sin hash no puede autenticarse: el estado «cuenta creada, aún no activada» es explícito.

STAND_SIZE_RULES · CLASIFICACIÓN			CREDENTIAL_RULES · CUOTA			
CATEGORÍA	MIN_M2	MAX_M2	CATEGORÍA	PER_BLOCK	BLOCK_M2	ROUNDING
Pequeño	5	12	Exhibitor	2	5	floor
Mediano	13	30	Guest	2	10	floor
Grande	31	50	Service	3	10	floor

25 m² → Exhibitor 10 · Guest 4 · Service 6
8 m² → Exhibitor 2 · Guest 0 · Service 0

El motor de reglas es un módulo puro —sin SQLAlchemy, sin FastAPI— que recibe las filas como parámetro y no las cachea: el cambio surte efecto en la siguiente petición. El test `test_changing_range_changes_classification` modifica un rango por SQL y comprueba que la clasificación cambia sola, sin desplegar nada.

Todo error viaja con un `code` estable y un `details` tipado. El frontend decide qué renderizar por ese código, nunca parseando el texto.

3 / 5

Ambigüedades detectadas

Ocho cuestiones que el enunciado deja abiertas y que no se pueden implementar sin decidir. Criterio general: **la opción que falla en voz alta gana a la que adivina en silencio.**

AMBIGÜEDAD	DECISIÓN	POR QUÉ
1 Redondeo. 13 m² con «2 por cada 5 m²»: ¿4 o 6?	floor, y el criterio es una columna (rounding_mode).	Es la lectura literal de «por cada N m²» y la conservadora para el organizador. Como es un dato y no código, otra feria decide distinto con un UPDATE.
2 Metrajes fuera de rango. Los rangos cubren 5–50 m². ¿Y 60?	Rechazar con STAND_SIZE_OUT_OF_RANGE.	Clasificar por aproximación crea datos erróneos silenciosos en la columna que alimenta el cupo de todo el stand. Un error explícito es recuperable; un dato mal clasificado, no.
3 ¿El representante consume credencial?	No. Si además opera el stand, se registra aparte como participante.	Separa la identidad de acceso al sistema de la credencial física. Un descuento invisible de cupo es imposible de explicar seis meses después.
4 ¿La cuota se congela al crear el expositor?	Cálculo derivado siempre. Si el metraje nuevo deja la cuota bajo lo ya asignado, se bloquea el cambio.	Una columna congelada se desincroniza en cuanto alguien corrige un metraje. El bloqueo convierte el único caso peligroso en un error legible.
5 ¿Cómo recibe la contraseña el usuario auto-creado?	No se envía ninguna: token de un solo uso, 72 h, guardado hasheado.	El correo es un canal persistente y sin cifrar en reposo. Una contraseña enviada por correo sigue ahí dentro de dos años; un token de un solo uso, no.
6 ¿La unicidad de identificación es global o por evento?	Por evento: UNIQUE(event_id, identification).	Las ediciones son independientes y la gente cambia de empleador. Global bloquearía registros legítimos y rompería el aislamiento entre ferias.
7 ¿La categoría de stand influye en la cuota? DETECTADA	No. Es informativa y de validación de rango.	Las cuotas están expresadas en m², no por categoría. Cruzarlas haría que ampliar un rango cambiara en silencio las cuotas de stands que nadie tocó.
8 Hay que notificar al participante, pero el correo no está entre sus datos requeridos. CONTRADICCIÓN	email opcional: si viene se notifica, si no, el alta no falla.	Exigirlo contradiría la lista de datos del propio enunciado y bloquearía altas legítimas. Notificar al representante se descartó: el enunciado nombra al participante.

Lo que se cuidó por debajo

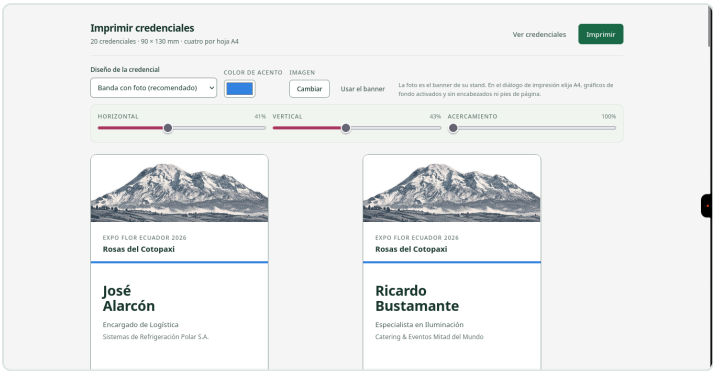
AUTORIZACIÓN A NIVEL DE OBJETO Un recurso ajeno responde 404 , no 403 : no se confirma que exista. No hay endpoint del representante que acepte un id de empresa; el scope sale del token.	CORREOS FUERA DE LA TRANSACCIÓN Salen después del COMMIT. Un SMTP caído registra el fallo y no tumba el alta —hay un test que lo demuestra—, y el admin puede reenviar el enlace.	IDENTIFICACIÓN VALIDADA DE VERDAD Cédula por módulo 10 y RUC por módulo 11 según el tercer dígito. Es la clave de deduplicación de todo el sistema, así que no basta con comprobar el largo.
---	--	---

Cada requisito evaluable tiene un test que lo demuestra

Bordes 12 / 13 / 30 / 31 m²	test_classify_boundaries	Aislamiento entre ediciones	test_same_identification_...
Metraje fuera de rango	test_classify_out_of_range	El cupo no se puede exceder	test_quota_exceeded
Redondeo floor: 13 m² → 4	test_quota_rounding_floor	IDOR: recurso ajeno → 404	test_representative_cannot_...
Rangos parametrizados	test_changing_range_...	Excel inválido: cero inserciones	test_invalid_rows_report_...
Duplicado en el mismo evento	test_duplicate_identification_...	Mailer caído no revierte el alta	test_mailer_failure_...

La interfaz también decide

Las mismas reglas del backend tienen que ser legibles en pantalla. Estas son las decisiones de producto que sostienen esa lectura.



Los gafetes se arman en el navegador. La tarjeta se describe en milímetros —90 × 130 mm, cuatro por hoja A4— y @media print esconde el marco de la aplicación: sin librería de PDF y sin render en servidor. El diseño, el color y el encuadre viajan en la URL, así que el enlace ya lleva puesto cómo se imprime.

ESTADOS VACÍOS, DE CARGA Y DE ERROR

Resueltos en todas las vistas. Una tabla vacía propone la acción siguiente en vez de quedarse en blanco, y el caso 0 / 0 —un stand pequeño sin cupo de Invitado— se dibuja bien en lugar de romperse.

UNA GUÍA QUE SE PUEDE VOLVER A VER

La primera visita de cada rol arranca un recorrido por la barra lateral, y queda disponible en el menú de opciones. Se guarda en el navegador, no en la base: es una preferencia de quien mira, no un dato de la feria.

EL ERROR DE DUPLICADO NO ES UN TOAST

Es un bloque que solo se cierra a mano, porque el usuario necesita leer y copiar *qué empresa* ya registró a esa persona antes de poder hacer nada.

EL CUPO SE VE ANTES DE ESCRIBIR

Cada opción del selector de categoría muestra cuántas quedan, y la agotada no se puede elegir. El tope se descubre al abrir el formulario, no al recibir un QUOTA_EXCEEDED con la ficha ya escrita. La validación autoritativa sigue siendo la del servidor.

NADA SE IMPORTA A CIEGAS

La carga masiva se lee en el navegador y se valida fila por fila contra el servidor con dry_run antes de confirmar. El mismo endpoint hace la vista previa y la importación: la única diferencia es el COMMIT.

SE INSTALA Y ABRE SIN CONEXIÓN

El frontend es una PWA: el service worker precachea el shell y la app abre offline con una pantalla de *sin conexión* en vez de un error del navegador. Las llamadas a /api quedan fuera de la caché: siempre van a la red, para no mostrar cupos ni credenciales viejos.



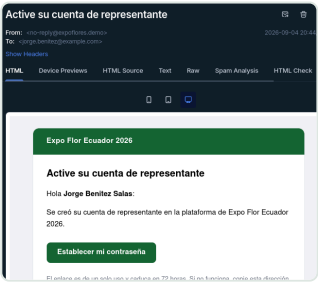
390 px. El panel se apila y el menú se contrae a iconos.



Sin scroll horizontal. Cada fila de la tabla se vuelve una ficha con sus etiquetas.



Accesibilidad. Sobre la base obligatoria, escala tipográfica, alto contraste y reducción de movimiento.



El correo de activación. Nunca viaja una contraseña: solo un enlace de un solo uso que caduca en 72 h.

Correos sin contraseñas

Salen **después del COMMIT**: un SMTP caído se registra y no tumba el alta. Al representante nunca se le manda una contraseña, sino un enlace de un solo uso, 72 h, guardado hasheado — el mismo que reenvía *¿Olvidaste tu contraseña?*, público y limitado como el login, con respuesta idéntica exista o no el correo. Al participante, solo si tiene correo (decisión 8).

Entrega continua

Cada push a main corre ruff, mpy - -strict y pytest **contra un Postgres 16 real**, más lint, tests y tsc del frontend. El deploy es otro job con needs: [backend, frontend]: si algo falla, no arranca. Dokploy reconstruye el mismo docker-compose.yml en un VPS propio y, como migración y seed son idempotentes, el demo se actualiza solo.

Qué cambiaría en producción

Refresh tokens y revocación: el JWT dura 60 min y no se invalida antes. **Correo y rate limiting fuera del proceso**: el reintento con backoff pide un worker; el contador, almacenamiento compartido. Y **row-level security** como segunda barrera del scope, con métricas y trazas sobre los logs JSON.

Lo que se dejó fuera a propósito: no hay colas, ni caché distribuida, ni microservicios, ni un design system propio. Robustez es hacer bien las piezas necesarias, no agregar piezas.